МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2025г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2025г

_____

(подпись)

Новосибирск 2025

Содержание

Введение 3
1. Подготовка сегментации 5
2. Включение RLS и политики 7
3. Передача контекста 9
4. Тесты доступов 11
5. Режим суперпользователя для auditor 15
Вывод 17
Приложение А 18
Приложение Б 25
Приложение В 34

Введение

Цель работы: реализовать построчную изоляцию данных между пользователями/группами/филиалами с помощью RLS.

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Подготовка сегментации. Выберите ось изоляции (SEG): организация/филиал/проект/группа и т.п. Добавьте segment_id во все ключевые таблицы app.*; FK на ref.segment (или аналогичный справочник). Создайте индексы, покрывающие условия политик (segment_id + ключи фильтрации).

2. Включение RLS и политики. Включите RLS и FORCE RLS на ключевых таблицах (ALTER TABLE ... ENABLE ROW LEVEL SECURITY). Опишите и создайте POLICY для SELECT/INSERT/UPDATE/DELETE с использованием current_user или безопасного GUC (например, app.user_id) для идентификации субъекта.

– Идентификация субъекта через current_user или безопасный GUC (например, current_setting('app.segment_id', true)), с резервным путём сопоставления логин-ролей → segment_id.

– Для INSERT/UPDATE задайте USING и WITH CHECK, запрещающие кросс-сегментные операции.

3. Передача контекста. SECURITY DEFINER-функция set_session_ctx(segment_id, actor_id), проверяющая право роли на сегмент и делающая SET LOCAL для GUC.

4. Тесты доступов (≥ 8 кейсов). Чтение «чужих» строк (должно быть пусто). Вставка/обновление с неверным segment_id (ошибка). Корректные операции в своём сегменте. Проверка работы set_session_ctx() – успешная и с ошибкой (если сегмент не принадлежит роли).

5. Режим суперпользователя для auditor. Создайте политику, которая позволяет роли auditor видеть все строки во всех сегментах (без ограничений RLS). Используйте условие USING (current_user = 'auditor' OR segment_id = current_setting(...)). Протестируйте – убедитесь, что auditor видит всё, а остальные – только своё.

1. Подготовка сегментации

В качестве оси изоляции данных были выбраны филиалы/студии. Каждый географический филиал студии дубляжа представляет собой отдельный сегмент данных. Все бизнес-данные: проекты, контракты, актёры, сессии записи и аудиоархивы привязаны к своему филиалу через поле segment_id. Пользователь может работать только с данными своего филиала. Данные других филиалов полностью невидимы и недоступны. Таблица сегментов представлена на рисунке 1.

segment_code	segment_name	studio_id	region	is_active
SEG-MSK	Филиал Москва (ЗвукЗапись)	1	Москва	t
SEG-SPB	Филиал Санкт-Петербург (ГолосДубляжа)	2	Санкт-Петербург	t
SEG-NSK	Филиал Новосибирск (АудиоМир)	3	Новосибирск	t
SEG-EKB	Филиал Екатеринбург (ВолнаЗвука)	4	Екатеринбург	t
SEG-KGD	Филиал Калининград (Синхрон)	5	Калининград	t
SEG-KYA	Филиал Красноярск (РечевойЦентр)	6	Красноярск	t
SEG-VVO	Филиал Владивосток (ЗвуковыеВолны)	7	Владивосток	t
SEG-SAM	Филиал Самара (Артикуляция)	8	Самара	t
SEG-NNV	Филиал Нижний Новгород (ГолосМастер)	9	Нижний Новгород	t
SEG-CHK	Филиал Челябинск (АудиоПрофи)	10	Челябинск	t

Рисунок 1 – Таблица сегментов

Для реализации сегментации была проведена модификация структуры базы данных. Во все ключевые таблицы бизнес-логики было добавлено поле segment_id, которое связывает каждую запись с конкретным сегментом-филиалом. Это включает таблицы проектов, контрактов, актёров, сессий записи, аудиоархивов и голосовых ролей. Каждое поле segment_id было объявлено обязательным для заполнения.

Для поддержания целостности данных были установлены внешние ключи от полей segment_id в таблицах приложения к справочнику сегментов.

Это предотвращает возможность создания записей с несуществующими сегментами и обеспечивает валидацию на уровне базы данных.

Для обеспечения высокой производительности запросов в условиях сегментированных данных были созданы специализированные индексы. Эти индексы покрывают типичные сценарии запросов, включающие фильтрацию по segment_id в сочетании с другими часто используемыми полями, такими как статусы проектов, даты сессий и идентификаторы актёров. Созданные индексы и их назначение представлены в таблице 1.

Таблица 1 – Индексы и назначение

Индекс	**Назначение**
idx_segments_code	Быстрый поиск сегмента по коду для установки контекста
idx_projects_segment_status	Фильтрация проектов по сегменту и статусу (открытые/закрытые)
idx_contracts_segment_actor	Поиск контрактов актёра в пределах сегмента
idx_sessions_segment_date	Фильтрация сессий по сегменту и дате проведения
idx_audio_segment_session	Быстрый доступ к аудиоархивам сессии в сегменте
idx_voice_roles_segment_project	Поиск голосовых ролей проекта в сегменте
idx_actors_segment	Фильтрация актёров по сегменту и идентификатору

Код реализации сегментации представлен в Приложении А.

2. Включение RLS и политики

На всех ключевых таблицах (actors, contracts, projects, recording_sessions, audio_archives, voice_roles) был активирован механизм построчного уровня безопасности (RLS). Особенностью реализации является использование принудительного режима безопасности (FORCE RLS), который гарантирует применение политик ко всем пользователям без исключений, включая владельцев таблиц и администраторов. Пример включения построчной безопасности представлен на рисунке 2.

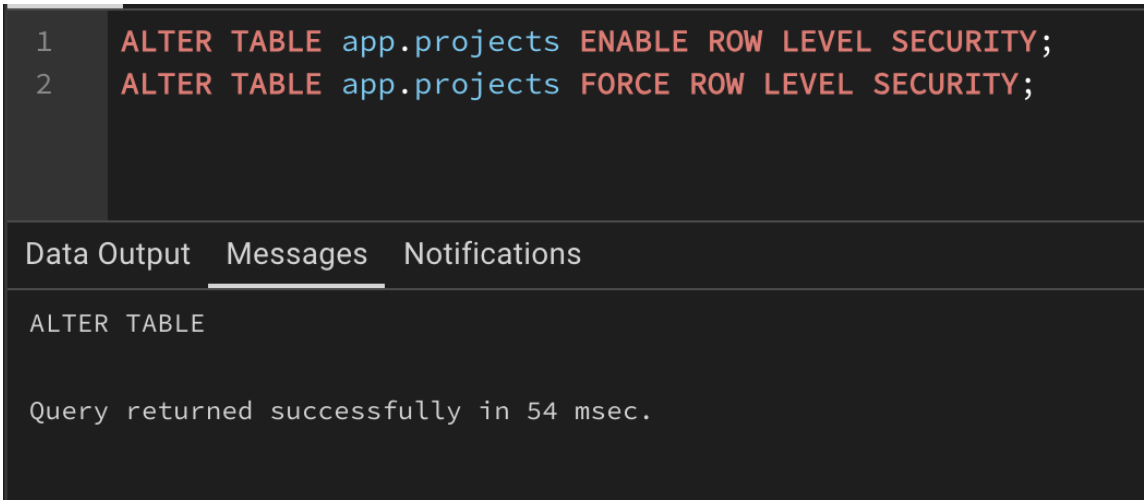


Рисунок 2 – Включение построчной безопасности

Все политики используют функцию app.segment_is_visible (p_segment_id), которая проверяет соответствие между текущим контекстом пользователя и сегментом данных. Функция представлена на рисунке 3.

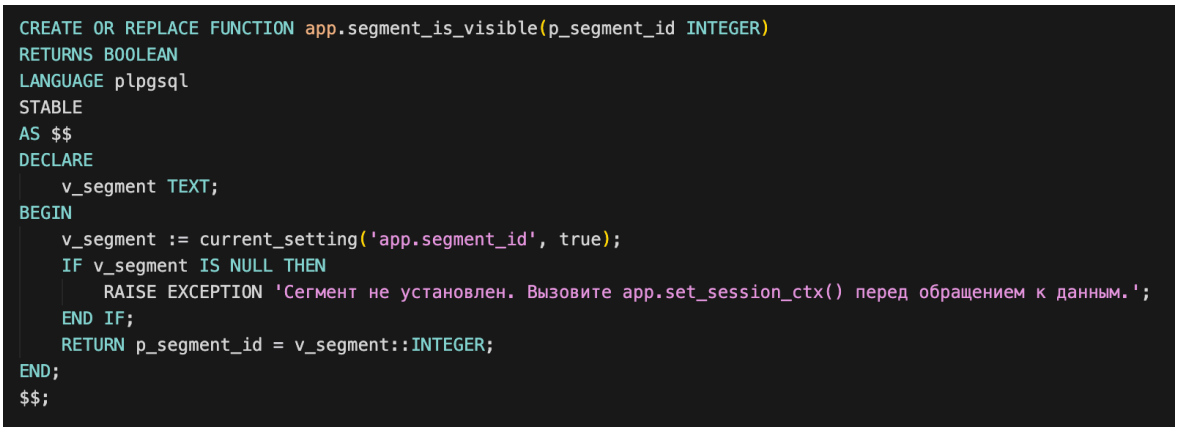


Рисунок 3 – Функция app.segment_is_visible(p_segment_id)

Для операций выборки (SELECT):

- Пользователь видит только записи своего сегмента;
- Используется условие USING с проверкой видимости сегмента.

Для операций модификации (INSERT/UPDATE/DELETE):

- USING - проверка отбора существующих записей;
- WITH CHECK - проверка новых значений при вставке/обновлении;
- Двойная проверка предотвращает кросс-сегментные операции.

Код включения RLS и политики представлен в Приложении Б.

3. Передача контекста

Была разработана безопасная функция set_session_ctx, которая выполняет роль шлюза для установки рабочего контекста пользователя, она устанавливает контекст сегмента и актёра для текущей транзакции. Функция реализована с правами SECURITY DEFINER, что обеспечивает выполнение с повышенными привилегиями при сохранении контроля доступа (Рисунок 4).

```
-- SECURITY DEFINER-функция передачи контекста
CREATE OR REPLACE FUNCTION app.set_session_ctx(p_segment_id INTEGER, p_actor_id INTEGER DEFAULT NULL)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
DECLARE
    v_effective_role TEXT;
BEGIN
    v_effective_role := current_setting('role', true);
    IF v_effective_role IS NULL THEN
        v_effective_role := session_user;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM app.segment_role_acl
        WHERE role_name = v_effective_role
          AND segment_id = p_segment_id
    ) THEN
        RAISE EXCEPTION 'Роль % не имеет доступа к сегменту %', v_effective_role, p_segment_id;
    END IF;

    PERFORM set_config('app.segment_id', p_segment_id::TEXT, FALSE);

    IF p_actor_id IS NULL THEN
        PERFORM set_config('app.actor_id', NULL, FALSE);
    ELSE
        IF NOT EXISTS (
            SELECT 1 FROM app.actors
            WHERE actor_id = p_actor_id
              AND segment_id = p_segment_id
        ) THEN
            RAISE EXCEPTION 'Актёр % не принадлежит сегменту %', p_actor_id, p_segment_id;
        END IF;
        PERFORM set_config('app.actor_id', p_actor_id::TEXT, FALSE);
    END IF;
END;
$$;
```

Рисунок 4 – Функция set_session_ctx

Перед установкой контекста функция выполняет проверку прав доступа. Она проверяет, имеет ли текущая роль пользователя право на работу с

запрашиваемым сегментом, используя таблицу соответствий ролей и сегментов. Дополнительно для роли актёра выполняется проверка принадлежности указанного актёра к запрашиваемому сегменту.

После успешной проверки прав доступа функция устанавливает параметры сессии, которые используются политиками RLS для фильтрации данных. Эти параметры действуют в пределах текущей транзакции и автоматически сбрасываются после её завершения. Код для создания передачи контекста представлен в Приложении Б.

4. Тесты доступов

Проверка изоляции данных между сегментами: app_writer, работающий в сегменте 1, не видит проекты из сегмента 2. При запросе возвращается пустой результат, что подтверждает полную изоляцию данных между разными филиалами (Рисунок 5).

```
NOTICE:  ТЕСТ 1: Проверка изоляции данных между сегментами
NOTICE:  Роль: app_writer, Сегмент: 1
NOTICE:  Ожидается: app_writer НЕ видит проекты из сегмента 2
DO
SET
 set_session_ctx
-----------------

(1 row)

NOTICE:  ✅ ТЕСТ 1 ПРОЙДЕН: Изоляция работает корректно
NOTICE:     • Всего проектов в БД: 2
NOTICE:     • Проектов из сегмента 2: 0 (недоступны)
DO
RESET
DO
```

Рисунок 5 – Тест 1

Запрет вставки данных в чужой сегмент: app_writer не может создать проект с segment_id = 2. Система RLS блокирует операцию вставки, возвращая ошибку безопасности (Рисунок 6).

```
NOTICE:  ТЕСТ 2: Запрет вставки данных в чужой сегмент
NOTICE:  Роль: app_writer, Сегмент: 1
NOTICE:  Ожидается: ошибка при попытке INSERT с segment_id = 2
SET
 set_session_ctx
-----------------

(1 row)

DO
NOTICE:  ✅ ТЕСТ 2 ПРОЙДЕН: RLS политика заблокировала вставку
NOTICE:     • Ошибка: new row violates row-level security policy for table "projects"
RESET
```

Рисунок 6 – Тест 2

Корректная вставка в свой сегмент: app_writer успешно создает проект в сегменте 1 с segment_id = 1. Операция завершается без ошибок, подтверждая, что авторизованные действия в пределах своего сегмента разрешены (Рисунок 7).

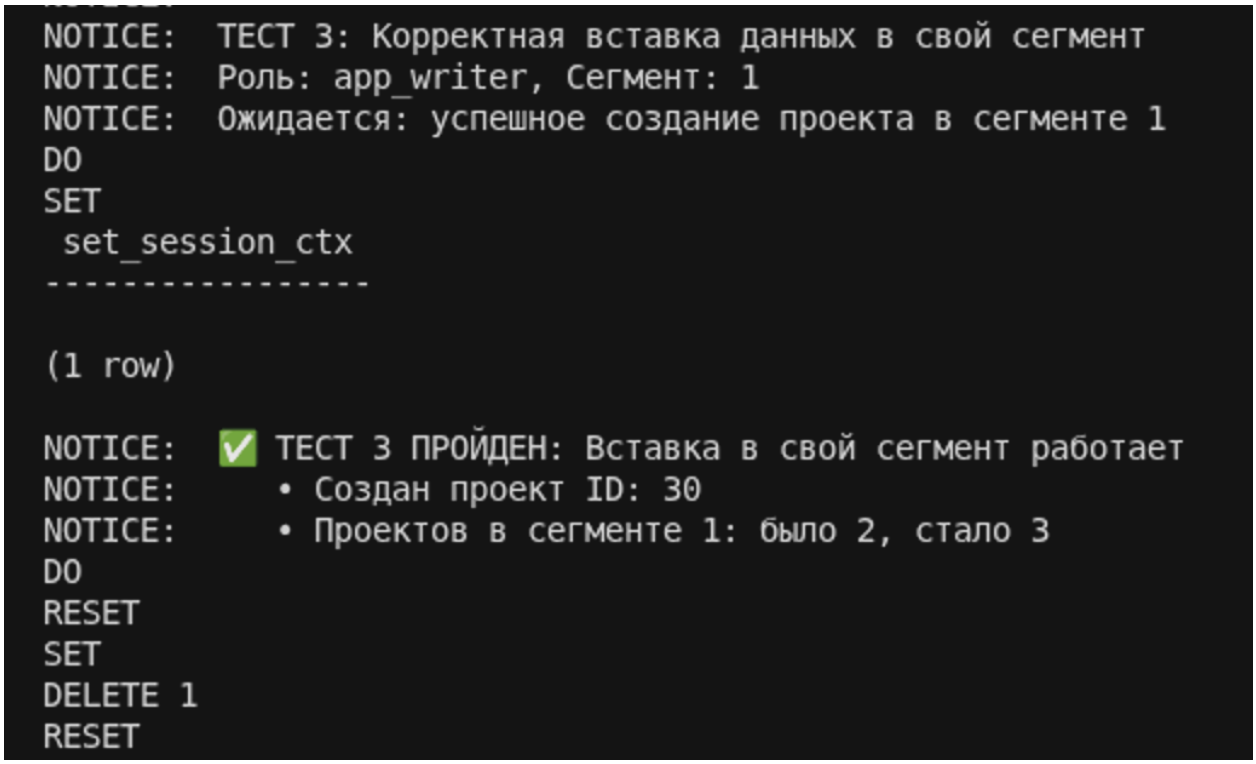


Рисунок 7 – Тест 3

Валидация доступа роли к сегменту: voice_actor не может установить контекст для сегмента 2. Функция set_session_ctx проверяет права доступа и блокирует попытку установки чужого сегмента (Рисунок 8).

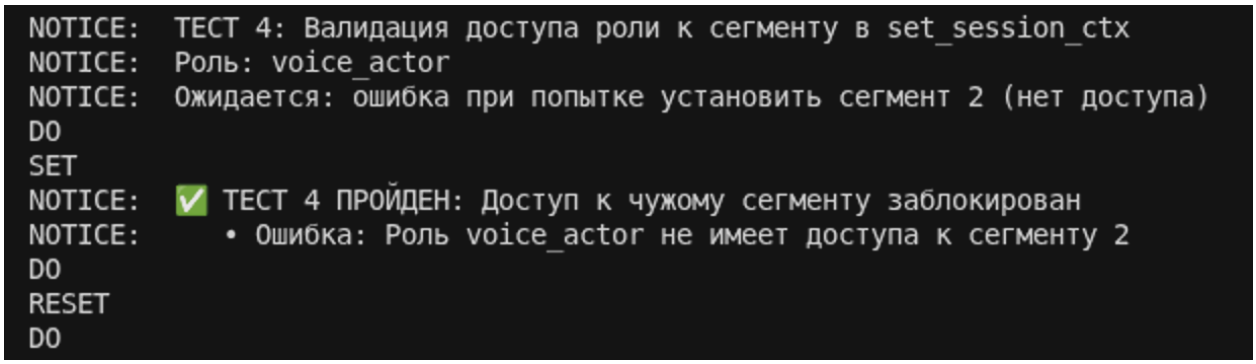


Рисунок 8 – Тест 4

Проверка принадлежности актёра сегменту: voice_actor не может установить несуществующий actor_id = 999 для сегмента 1. Система проверяет соответствие актёра указанному сегменту (Рисунок 9).

```
NOTICE:  ТЕСТ 5: Проверка принадлежности актёра сегменту
NOTICE:  Роль: voice_actor, Сегмент: 1, Actor ID: 999 (не существует)
NOTICE:  Ожидается: ошибка при установке несуществующего actor_id
SET
NOTICE:  ✅ ТЕСТ 5 ПРОЙДЕН: Валидация актёра работает
NOTICE:     • Ошибка: Актёр 999 не принадлежит сегменту 1
DO
 set_session_ctx
-----------------

(1 row)

RESET
```

Рисунок 9 – Тест 5

Обновление данных в своём сегменте: успешно меняет статус проекта в сегменте 1. Операция обновления работает корректно в пределах разрешённого сегмента (Рисунок 10).

```
NOTICE:  ТЕСТ 6: Обновление данных в своём сегменте
NOTICE:  Роль: studio_manager, Сегмент: 1
NOTICE:  Ожидается: успешное обновление проекта в сегменте 1
DO
SET
 set_session_ctx
-----------------

(1 row)

NOTICE:  ✅ ТЕСТ 6 ПРОЙДЕН: Обновление в своём сегменте работает
NOTICE:     • Обновлён проект ID: 1
NOTICE:     • Статус изменён: active → in_review
DO
RESET
```

Рисунок 10 – Тест 6

Запрет подмены segment_id: нельзя изменить segment_id проекта с 1 на 2. RLS политика блокирует попытку переноса данных в чужой сегмент через операцию UPDATE (Рисунок 11).

```
NOTICE:  ТЕСТ 7: Запрет подмены segment_id при обновлении
NOTICE:  Роль: studio_manager, Сегмент: 1
NOTICE:  Ожидается: ошибка при попытке изменить segment_id на 2
DO
SET
 set_session_ctx
-----------------

(1 row)

DO
RESET
DO
SET
NOTICE:  ✅ ТЕСТ 7 ПРОЙДЕН: Подмена сегмента заблокирована
NOTICE:     • Проект ID: 1
NOTICE:     • Ошибка: new row violates row-level security policy for table "projects"
```

Рисунок 11 – Тест 7

Доступ voice_actor к своим контрактам: voice_actor с actor_id = 1 видит только свои контракты в сегменте 1. Двойная фильтрация по segment_id и actor_id обеспечивает точный доступ только к связанным данным (Рисунок 12).

```
NOTICE:  ТЕСТ 8: Доступ voice_actor к своим контрактам
NOTICE:  Роль: voice_actor, Сегмент: 1, Actor ID: 1
NOTICE:  Ожидается: voice_actor видит только свои контракты в сегменте 1
DO
SET
 set_session_ctx
-----------------

(1 row)

NOTICE:  ✅ ТЕСТ 8 ПРОЙДЕН: voice_actor получил доступ к своим данным
NOTICE:     • Актёр: Иван Петров (ID: 1)
NOTICE:     • Доступно контрактов: 3
NOTICE:     • Доступно актёров: 1
DO
RESET
```

Рисунок 12 – Тест 8

Код тестирования представлен в Приложении В.

5. Режим суперпользователя для auditor

Реализована функция «app.segment_filter()», которая содержит логику для роли auditor. При проверке доступа к сегменту функция возвращает «TRUE» для пользователя с ролью auditor, полностью отключая для него механизм сегментации. Для всех остальных ролей применяется стандартная проверка видимости сегмента через функцию «app.segment_is_visible()» (Рисунок 13).

```
-- Универсальная проверка сегмента для политик (с режимом auditor)
CREATE OR REPLACE FUNCTION app.segment_filter(p_segment_id INTEGER)
RETURNS BOOLEAN
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    IF current_user = 'auditor' THEN
        RETURN TRUE;
    END IF;
    RETURN app.segment_is_visible(p_segment_id);
END;
$$;
```

Рисунок 13 – Функция app.segment_filter()

В каждой RLS-политике SELECT для всех ключевых таблиц системы добавлено условие current_user = auditor, которое позволяет обходить ограничения построчной безопасности. На рисунке 14 представлен один из примеров.

```
CREATE POLICY seg_projects_select ON app.projects
    FOR SELECT
    TO app_reader, app_writer, studio_manager, voice_actor, auditor
    USING (current_user = 'auditor' OR app.segment_is_visible(segment_id));
```

Рисунок 14 – Дополнительно условие

Проведено тестирование доступа auditor ко всем таблицам системы. Auditor получает полный доступ на чтение без установки контекста и видит данные из всех сегментов одновременно.

```
NOTICE:  ТЕСТ 9: Режим аудитора - полный доступ ко всем данным
NOTICE:  Роль: auditor (БЕЗ set_session_ctx)
NOTICE:  Ожидается: auditor видит ВСЕ данные из ВСЕХ сегментов
DO
SET
NOTICE:  ТЕСТ 9 ПРОЙДЕН: Auditor имеет полный доступ ко всем данным
NOTICE:
NOTICE:
NOTICE:      | Таблица                | Сегментов | Всего записей |
NOTICE:      |------------------------|-----------|---------------|
NOTICE:      | app.projects           | 5         | 10            |
NOTICE:      | app.contracts          | 3         | 13            |
NOTICE:      | app.actors             | 3         | 10            |
NOTICE:      | app.recording_sessions | 3         | 11            |
NOTICE:      | app.audio_archives     | 3         | 11            |
NOTICE:      | app.voice_roles        | 3         | 10            |
NOTICE:
NOTICE:
NOTICE:      Auditor видит данные из 5 сегментов БЕЗ ограничений RLS
DO
RESET
```

Рисунок 15 – Тестирование auditor

Тесты подтверждают отключение RLS-ограничений, auditor просматривает статистику по проектам, контрактам, актёрам, сессиям записи, аудиоархивам и голосовым ролям без каких-либо ограничений.

Вывод

В ходе выполнения лабораторной работы для системы управления студией дубляжа была успешно разработана и внедрена комплексная система безопасности на основе построчного уровня доступа (Row Level Security).

Основным достижением работы стала реализация сегментации данных по географическому принципу, где каждый филиал студии представляет собой изолированный сегмент данных. В рамках подготовки сегментации были модифицированы все ключевые таблицы бизнес-логики: projects, contracts, actors, recording_sessions, audio_archives и voice_roles, с добавлением обязательного поля segment_id и созданием индексов для оптимизации производительности.

Особенностью реализации стало применение принудительного режима безопасности (FORCE RLS), который гарантирует применение политик ко всем пользователям без исключений. Были разработаны универсальные политики доступа, использующие функцию app.segment_is_visible() для проверки соответствия между текущим контекстом пользователя и сегментом данных. Для операций модификации реализована двойная проверка через условия USING и WITH CHECK, что полностью исключает возможность кросс-сегментных операций.

Важным элементом системы стала SECURITY DEFINER функция set_session_ctx(), обеспечивающая безопасную передачу контекста доступа с проверкой прав роли на запрашиваемый сегмент и валидацией принадлежности актёров к сегментам. Функция устанавливает параметры сессии в пределах транзакции с автоматическим сбросом, что предотвращает утечки контекста.

Проведенное тестирование подтвердило эффективность системы безопасности. Отдельно реализован режим аудитора с полным доступом на чтение ко всем данным системы для мониторинга и анализа.

Приложение А

```
-- lr3_01_segment_model.sql
-- Лабораторная работа №3: подготовка сегментации и перенос данных

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
   RAISE NOTICE 'LR3: начало настройки сегментации данных';
END $$;

-- 1. Справочник сегментов (ось изоляции — филиал / студия)
CREATE TABLE IF NOT EXISTS ref.segments (
   segment_id SERIAL PRIMARY KEY,
   segment_code VARCHAR(30) UNIQUE NOT NULL,
   segment_name TEXT NOT NULL,
   studio_id INTEGER NOT NULL UNIQUE REFERENCES
ref.studios(studio_id),
   region VARCHAR(100),
   is_active BOOLEAN DEFAULT TRUE,
   created_at TIMESTAMPTZ DEFAULT NOW()
);

COMMENT ON TABLE ref.segments IS 'Сегменты (филиалы/студии) для
RLS-изоляции';
COMMENT ON COLUMN ref.segments.segment_code IS 'Уникальный код
сегмента для SET LOCAL app.segment_id';

INSERT INTO ref.segments (segment_code, segment_name, studio_id, region)
VALUES
   ('SEG-MSK', 'Филиал Москва (ЗвукЗапись)', 1, 'Москва'),
   ('SEG-SPB', 'Филиал Санкт-Петербург (ГолосДубляжа)', 2, 'Санкт-
Петербург'),
   ('SEG-NSK', 'Филиал Новосибирск (АудиоМир)', 3, 'Новосибирск'),
   ('SEG-EKB', 'Филиал Екатеринбург (ВолнаЗвука)', 4, 'Екатеринбург'),
   ('SEG-KGD', 'Филиал Калининград (Синхрон)', 5, 'Калининград'),
   ('SEG-KYA', 'Филиал Красноярск (РечевойЦентр)', 6, 'Красноярск'),
   ('SEG-VVO', 'Филиал Владивосток (ЗвуковыеВолны)', 7, 'Владивосток'),
   ('SEG-SAM', 'Филиал Самара (Артикуляция)', 8, 'Самара'),
   ('SEG-NNV', 'Филиал Нижний Новгород (ГолосМастер)', 9, 'Нижний
Новгород'),
   ('SEG-CHK', 'Филиал Челябинск (АудиоПрофи)', 10, 'Челябинск')
```

```
ON CONFLICT (studio_id) DO UPDATE
   SET segment_code = EXCLUDED.segment_code,
      segment_name = EXCLUDED.segment_name,
      region = EXCLUDED.region,
      is_active = TRUE;

CREATE INDEX IF NOT EXISTS idx_segments_code ON
ref.segments(segment_code);

-- 2. ACL сегментов для ролей
CREATE TABLE IF NOT EXISTS app.segment_role_acl (
   acl_id SERIAL PRIMARY KEY,
   role_name TEXT NOT NULL,
   segment_id INTEGER NOT NULL REFERENCES ref.segments(segment_id)
ON DELETE CASCADE,
   can_manage BOOLEAN DEFAULT FALSE,
   UNIQUE(role_name, segment_id)
);

COMMENT ON TABLE app.segment_role_acl IS 'Допустимые соответствия
роль ↔ сегмент для установки контекста RLS';

INSERT INTO app.segment_role_acl (role_name, segment_id, can_manage)
VALUES
   ('studio_manager', 1, TRUE),
   ('studio_manager', 2, FALSE),
   ('app_writer', 1, TRUE),
   ('app_writer', 2, TRUE),
   ('app_writer', 3, TRUE),
   ('app_reader', 1, FALSE),
   ('app_reader', 2, FALSE),
   ('voice_actor', 1, FALSE)
ON CONFLICT DO NOTHING;

-- 3. Добавление segment_id в ключевые таблицы app.*

-- helper to enforce FK after backfill
DO $$
BEGIN
   IF NOT EXISTS (
      SELECT 1 FROM information_schema.columns
      WHERE table_schema = 'app' AND table_name = 'projects' AND
column_name = 'segment_id'
   ) THEN
      ALTER TABLE app.projects ADD COLUMN segment_id INTEGER;
```

```
    END IF;

    UPDATE app.projects p
    SET segment_id = s.segment_id
    FROM ref.segments s
    WHERE p.segment_id IS NULL
      AND s.studio_id = p.studio_id;

    ALTER TABLE app.projects
        ALTER COLUMN segment_id SET NOT NULL;

    BEGIN
        ALTER TABLE app.projects
            ADD CONSTRAINT projects_segment_fk FOREIGN KEY (segment_id)
REFERENCES ref.segments(segment_id);
    EXCEPTION WHEN duplicate_object THEN
        NULL;
    END;
END $$;

CREATE INDEX IF NOT EXISTS idx_projects_segment_status ON
app.projects(segment_id, status);

DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_schema = 'app' AND table_name = 'contracts' AND
column_name = 'segment_id'
    ) THEN
        ALTER TABLE app.contracts ADD COLUMN segment_id INTEGER;
    END IF;

    UPDATE app.contracts c
    SET segment_id = p.segment_id
    FROM app.projects p
    WHERE c.segment_id IS NULL
      AND c.project_id = p.project_id;

    ALTER TABLE app.contracts
        ALTER COLUMN segment_id SET NOT NULL;

    BEGIN
        ALTER TABLE app.contracts
```

```
        ADD CONSTRAINT contracts_segment_fk FOREIGN KEY (segment_id)
REFERENCES ref.segments(segment_id);
   EXCEPTION WHEN duplicate_object THEN
      NULL;
   END;
END $$;

CREATE INDEX IF NOT EXISTS idx_contracts_segment_actor ON
app.contracts(segment_id, actor_id);

DO $$
BEGIN
   IF NOT EXISTS (
      SELECT 1 FROM information_schema.columns
      WHERE table_schema = 'app' AND table_name = 'recording_sessions' AND
column_name = 'segment_id'
   ) THEN
      ALTER TABLE app.recording_sessions ADD COLUMN segment_id
INTEGER;
   END IF;

   UPDATE app.recording_sessions rs
   SET segment_id = c.segment_id
   FROM app.contracts c
   WHERE rs.segment_id IS NULL
     AND rs.contract_id = c.contract_id;

   ALTER TABLE app.recording_sessions
      ALTER COLUMN segment_id SET NOT NULL;

   BEGIN
      ALTER TABLE app.recording_sessions
         ADD CONSTRAINT sessions_segment_fk FOREIGN KEY (segment_id)
REFERENCES ref.segments(segment_id);
   EXCEPTION WHEN duplicate_object THEN
      NULL;
   END;
END $$;

CREATE INDEX IF NOT EXISTS idx_sessions_segment_date ON
app.recording_sessions(segment_id, session_date);

DO $$
BEGIN
   IF NOT EXISTS (
```

```
      SELECT 1 FROM information_schema.columns
      WHERE table_schema = 'app' AND table_name = 'audio_archives' AND
column_name = 'segment_id'
   ) THEN
      ALTER TABLE app.audio_archives ADD COLUMN segment_id INTEGER;
   END IF;

   UPDATE app.audio_archives aa
   SET segment_id = rs.segment_id
   FROM app.recording_sessions rs
   WHERE aa.segment_id IS NULL
     AND aa.session_id = rs.session_id;

   ALTER TABLE app.audio_archives
      ALTER COLUMN segment_id SET NOT NULL;

   BEGIN
      ALTER TABLE app.audio_archives
         ADD CONSTRAINT audio_segment_fk FOREIGN KEY (segment_id)
REFERENCES ref.segments(segment_id);
   EXCEPTION WHEN duplicate_object THEN
      NULL;
   END;
END $$;

CREATE INDEX IF NOT EXISTS idx_audio_segment_session ON
app.audio_archives(segment_id, session_id);

DO $$
BEGIN
   IF NOT EXISTS (
      SELECT 1 FROM information_schema.columns
      WHERE table_schema = 'app' AND table_name = 'voice_roles' AND
column_name = 'segment_id'
   ) THEN
      ALTER TABLE app.voice_roles ADD COLUMN segment_id INTEGER;
   END IF;

   UPDATE app.voice_roles vr
   SET segment_id = p.segment_id
   FROM app.projects p
   WHERE vr.segment_id IS NULL
     AND vr.project_id = p.project_id;

   ALTER TABLE app.voice_roles
```

```
      ALTER COLUMN segment_id SET NOT NULL;

  BEGIN
    ALTER TABLE app.voice_roles
      ADD CONSTRAINT voice_roles_segment_fk FOREIGN KEY
(segment_id) REFERENCES ref.segments(segment_id);
  EXCEPTION WHEN duplicate_object THEN
    NULL;
  END;
END $$;

CREATE INDEX IF NOT EXISTS idx_voice_roles_segment_project ON
app.voice_roles(segment_id, project_id);

DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM information_schema.columns
    WHERE table_schema = 'app' AND table_name = 'actors' AND
column_name = 'segment_id'
  ) THEN
    ALTER TABLE app.actors ADD COLUMN segment_id INTEGER;
  END IF;

  UPDATE app.actors a
  SET segment_id = (
    SELECT c.segment_id
    FROM app.contracts c
    WHERE c.actor_id = a.actor_id
    ORDER BY c.contract_id
    LIMIT 1
  )
  WHERE a.segment_id IS NULL;

  UPDATE app.actors
  SET segment_id = 1
  WHERE segment_id IS NULL;

  ALTER TABLE app.actors
    ALTER COLUMN segment_id SET NOT NULL;

  BEGIN
    ALTER TABLE app.actors
      ADD CONSTRAINT actors_segment_fk FOREIGN KEY (segment_id)
REFERENCES ref.segments(segment_id);
```

```
    EXCEPTION WHEN duplicate_object THEN
      NULL;
    END;
END $$;

CREATE INDEX IF NOT EXISTS idx_actors_segment ON
app.actors(segment_id, actor_id);

DO $$
BEGIN
    RAISE NOTICE 'LR3: сегментация успешно применена';
END $$;

RESET ROLE;
```

Приложение Б

```
-- lr3_02_rls.sql
-- Лабораторная работа №3: RLS политики и передача контекста

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
   RAISE NOTICE 'LR3: начало перенастройки RLS и функций контекста';
END $$;

-- Удаляем устаревшие политики ЛР1/ЛР2
DO $$
DECLARE
   r RECORD;
BEGIN
   FOR r IN
      SELECT policyname, tablename
      FROM pg_policies
      WHERE schemaname = 'app'
        AND policyname IN (
           'actor_self_select', 'dml_admin_actors',
           'actor_own_contracts', 'dml_admin_contracts',
           'actor_own_sessions', 'dml_admin_sessions',
           'studio_own_projects', 'dml_admin_projects',
           'hide_salaries', 'dml_admin_audio_archives'
        )
   LOOP
      EXECUTE format('DROP POLICY IF EXISTS %I ON app.%I',
r.policyname, r.tablename);
   END LOOP;
END $$;

-- Включаем (повторно) RLS и FORCE на ключевых таблицах
ALTER TABLE app.actors ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.recording_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.audio_archives ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.voice_roles ENABLE ROW LEVEL SECURITY;

ALTER TABLE app.actors FORCE ROW LEVEL SECURITY;
```

```
ALTER TABLE app.contracts FORCE ROW LEVEL SECURITY;
ALTER TABLE app.projects FORCE ROW LEVEL SECURITY;
ALTER TABLE app.recording_sessions FORCE ROW LEVEL SECURITY;
ALTER TABLE app.audio_archives FORCE ROW LEVEL SECURITY;
ALTER TABLE app.voice_roles FORCE ROW LEVEL SECURITY;

-- Вспомогательные функции
DROP FUNCTION IF EXISTS app.get_current_user_studio();

CREATE OR REPLACE FUNCTION app.segment_is_visible(p_segment_id
INTEGER)
RETURNS BOOLEAN
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
   v_segment TEXT;
BEGIN
   v_segment := current_setting('app.segment_id', true);
   IF v_segment IS NULL THEN
      RAISE EXCEPTION 'Сегмент не установлен. Вызовите
app.set_session_ctx() перед обращением к данным.';
   END IF;
   RETURN p_segment_id = v_segment::INTEGER;
END;
$$;

CREATE OR REPLACE FUNCTION app.actor_scope_allows(p_actor_id
INTEGER)
RETURNS BOOLEAN
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
   v_actor TEXT;
BEGIN
   IF current_user <> 'voice_actor' THEN
      RETURN TRUE;
   END IF;

   v_actor := current_setting('app.actor_id', true);
   IF v_actor IS NULL THEN
      RAISE EXCEPTION 'Для роли voice_actor необходимо установить
actor_id через app.set_session_ctx().';
   END IF;
```

```
    RETURN p_actor_id = v_actor::INTEGER;
END;
$$;

-- SECURITY DEFINER-функция передачи контекста
CREATE OR REPLACE FUNCTION app.set_session_ctx(p_segment_id
INTEGER, p_actor_id INTEGER DEFAULT NULL)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
DECLARE
    v_effective_role TEXT;
BEGIN
    v_effective_role := current_setting('role', true);
    IF v_effective_role IS NULL THEN
        v_effective_role := session_user;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM app.segment_role_acl
        WHERE role_name = v_effective_role
          AND segment_id = p_segment_id
    ) THEN
        RAISE EXCEPTION 'Роль % не имеет доступа к сегменту %',
v_effective_role, p_segment_id;
    END IF;

    PERFORM set_config('app.segment_id', p_segment_id::TEXT, FALSE);

    IF p_actor_id IS NULL THEN
        PERFORM set_config('app.actor_id', NULL, FALSE);
    ELSE
        IF NOT EXISTS (
            SELECT 1 FROM app.actors
            WHERE actor_id = p_actor_id
              AND segment_id = p_segment_id
        ) THEN
            RAISE EXCEPTION 'Актёр % не принадлежит сегменту %', p_actor_id,
p_segment_id;
        END IF;
        PERFORM set_config('app.actor_id', p_actor_id::TEXT, FALSE);
```

```
    END IF;
END;
$$;

COMMENT ON FUNCTION app.set_session_ctx(INTEGER, INTEGER) IS
'Устанавливает безопасный контекст сегмента и актёра для политик RLS
(SEG + actor)';

GRANT EXECUTE ON FUNCTION app.set_session_ctx(INTEGER, INTEGER)
TO app_reader, app_writer, studio_manager, voice_actor;

-- Универсальная проверка сегмента для политик (с режимом auditor)
CREATE OR REPLACE FUNCTION app.segment_filter(p_segment_id
INTEGER)
RETURNS BOOLEAN
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    IF current_user = 'auditor' THEN
        RETURN TRUE;
    END IF;
    RETURN app.segment_is_visible(p_segment_id);
END;
$$;

-- Политики для app.projects
DROP POLICY IF EXISTS seg_projects_select ON app.projects;
DROP POLICY IF EXISTS seg_projects_insert ON app.projects;
DROP POLICY IF EXISTS seg_projects_update ON app.projects;
DROP POLICY IF EXISTS seg_projects_delete ON app.projects;
DROP POLICY IF EXISTS dml_admin_projects_all ON app.projects;

CREATE POLICY seg_projects_select ON app.projects
    FOR SELECT
    TO app_reader, app_writer, studio_manager, voice_actor, auditor
    USING (current_user = 'auditor' OR app.segment_is_visible(segment_id));

CREATE POLICY seg_projects_insert ON app.projects
    FOR INSERT
    TO app_writer, studio_manager
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_projects_update ON app.projects
    FOR UPDATE
```

```
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id))
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_projects_delete ON app.projects
    FOR DELETE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id));

CREATE POLICY dml_admin_projects_all ON app.projects
    FOR ALL
    TO dml_admin
    USING (TRUE);

-- Политики для app.contracts
DROP POLICY IF EXISTS seg_contracts_select ON app.contracts;
DROP POLICY IF EXISTS seg_contracts_insert ON app.contracts;
DROP POLICY IF EXISTS seg_contracts_update ON app.contracts;
DROP POLICY IF EXISTS seg_contracts_delete ON app.contracts;
DROP POLICY IF EXISTS dml_admin_contracts_all ON app.contracts;

CREATE POLICY seg_contracts_select ON app.contracts
    FOR SELECT
    TO app_reader, app_writer, studio_manager, voice_actor, auditor
    USING (
        current_user = 'auditor'
        OR (app.segment_is_visible(segment_id) AND
app.actor_scope_allows(actor_id))
    );

CREATE POLICY seg_contracts_insert ON app.contracts
    FOR INSERT
    TO app_writer, studio_manager
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_contracts_update ON app.contracts
    FOR UPDATE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id))
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_contracts_delete ON app.contracts
    FOR DELETE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id));
```

```
CREATE POLICY dml_admin_contracts_all ON app.contracts
   FOR ALL
   TO dml_admin
   USING (TRUE);

-- Политики для app.actors
DROP POLICY IF EXISTS seg_actors_select ON app.actors;
DROP POLICY IF EXISTS seg_actors_update ON app.actors;
DROP POLICY IF EXISTS dml_admin_actors_all ON app.actors;

CREATE POLICY seg_actors_select ON app.actors
   FOR SELECT
   TO app_reader, app_writer, studio_manager, voice_actor, auditor
   USING (
      current_user = 'auditor'
      OR (app.segment_is_visible(segment_id) AND
app.actor_scope_allows(actor_id))
   );

CREATE POLICY seg_actors_update ON app.actors
   FOR UPDATE
   TO app_writer, studio_manager
   USING (app.segment_is_visible(segment_id))
   WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY dml_admin_actors_all ON app.actors
   FOR ALL
   TO dml_admin
   USING (TRUE);

-- Политики для app.recording_sessions
DROP POLICY IF EXISTS seg_sessions_select ON app.recording_sessions;
DROP POLICY IF EXISTS seg_sessions_insert ON app.recording_sessions;
DROP POLICY IF EXISTS seg_sessions_update ON app.recording_sessions;
DROP POLICY IF EXISTS seg_sessions_delete ON app.recording_sessions;
DROP POLICY IF EXISTS dml_admin_sessions_all ON app.recording_sessions;

CREATE POLICY seg_sessions_select ON app.recording_sessions
   FOR SELECT
   TO app_reader, app_writer, studio_manager, voice_actor, auditor
   USING (current_user = 'auditor' OR app.segment_is_visible(segment_id));

CREATE POLICY seg_sessions_insert ON app.recording_sessions
   FOR INSERT
```

```
    TO app_writer, studio_manager
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_sessions_update ON app.recording_sessions
    FOR UPDATE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id))
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_sessions_delete ON app.recording_sessions
    FOR DELETE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id));

CREATE POLICY dml_admin_sessions_all ON app.recording_sessions
    FOR ALL
    TO dml_admin
    USING (TRUE);

-- Политики для app.audio_archives
DROP POLICY IF EXISTS seg_audio_select ON app.audio_archives;
DROP POLICY IF EXISTS seg_audio_insert ON app.audio_archives;
DROP POLICY IF EXISTS seg_audio_update ON app.audio_archives;
DROP POLICY IF EXISTS seg_audio_delete ON app.audio_archives;
DROP POLICY IF EXISTS dml_admin_audio_all ON app.audio_archives;

CREATE POLICY seg_audio_select ON app.audio_archives
    FOR SELECT
    TO app_reader, app_writer, studio_manager, voice_actor, auditor
    USING (current_user = 'auditor' OR app.segment_is_visible(segment_id));

CREATE POLICY seg_audio_insert ON app.audio_archives
    FOR INSERT
    TO app_writer, studio_manager
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_audio_update ON app.audio_archives
    FOR UPDATE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id))
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_audio_delete ON app.audio_archives
    FOR DELETE
    TO app_writer, studio_manager
```

```
    USING (app.segment_is_visible(segment_id));

CREATE POLICY dml_admin_audio_all ON app.audio_archives
    FOR ALL
    TO dml_admin
    USING (TRUE);

-- Политики для app.voice_roles
DROP POLICY IF EXISTS seg_voice_roles_select ON app.voice_roles;
DROP POLICY IF EXISTS seg_voice_roles_insert ON app.voice_roles;
DROP POLICY IF EXISTS seg_voice_roles_update ON app.voice_roles;
DROP POLICY IF EXISTS seg_voice_roles_delete ON app.voice_roles;
DROP POLICY IF EXISTS dml_admin_voice_roles_all ON app.voice_roles;

CREATE POLICY seg_voice_roles_select ON app.voice_roles
    FOR SELECT
    TO app_reader, app_writer, studio_manager, voice_actor, auditor
    USING (current_user = 'auditor' OR app.segment_is_visible(segment_id));

CREATE POLICY seg_voice_roles_insert ON app.voice_roles
    FOR INSERT
    TO app_writer, studio_manager
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_voice_roles_update ON app.voice_roles
    FOR UPDATE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id))
    WITH CHECK (app.segment_is_visible(segment_id));

CREATE POLICY seg_voice_roles_delete ON app.voice_roles
    FOR DELETE
    TO app_writer, studio_manager
    USING (app.segment_is_visible(segment_id));

CREATE POLICY dml_admin_voice_roles_all ON app.voice_roles
    FOR ALL
    TO dml_admin
    USING (TRUE);

DO $$
BEGIN
    RAISE NOTICE 'LR3: политики RLS и функция app.set_session_ctx
успешно обновлены';
END $$;
```

```
RESET ROLE;
```

Приложение В

```
-- lr3_03_tests.sql

SET search_path TO app, ref, audit, stg, public;

-------------------------------------------------------------
-- Тест 1. Чтение чужого сегмента возвращает пустой набор
-------------------------------------------------------------
DO $$
BEGIN
   RAISE NOTICE '━━━
```

```
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '━━━
```

```
    RAISE NOTICE '━━━
```

```
DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '━━━
```

```
    RAISE NOTICE 'ТЕСТ 5: Проверка принадлежности актёра сегменту';
    RAISE NOTICE 'Роль: voice_actor, Сегмент: 1, Actor ID: 999 (не
существует)';
    RAISE NOTICE 'Ожидается: ошибка при установке несуществующего
actor_id';
END $$;

SET ROLE voice_actor;

DO $$
BEGIN
    BEGIN
        PERFORM app.set_session_ctx(1, 999);
        RAISE EXCEPTION '❌ ТЕСТ 5 ПРОВАЛЕН: установлен actor_id,
которого нет в сегменте';
    EXCEPTION WHEN others THEN
        IF SQLERRM LIKE '%не принадлежит сегменту%' THEN
            RAISE NOTICE '✅ ТЕСТ 5 ПРОЙДЕН: Валидация актёра работает';
            RAISE NOTICE '   • Ошибка: %', SQLERRM;
        ELSE
            RAISE EXCEPTION '❌ ТЕСТ 5: Неожиданная ошибка: %',
SQLERRM;
        END IF;
    END;
END $$;

-- Корректная установка контекста для дальнейших тестов
SELECT app.set_session_ctx(1, 1);
RESET ROLE;

------------------------------------------------------------
-- Тест 6. studio_manager успешно обновляет данные своего сегмента
------------------------------------------------------------
DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
    RAISE NOTICE 'ТЕСТ 6: Обновление данных в своём сегменте';
    RAISE NOTICE 'Роль: studio_manager, Сегмент: 1';
    RAISE NOTICE 'Ожидается: успешное обновление проекта в сегменте 1';
END $$;
```

```
SET ROLE studio_manager;
SELECT app.set_session_ctx(1, NULL);

DO $$
DECLARE
    v_project_id INTEGER;
    v_old_status TEXT;
    v_new_status TEXT := 'in_review';
BEGIN
    SELECT project_id, status INTO v_project_id, v_old_status
    FROM app.projects
    WHERE segment_id = 1
    ORDER BY project_id
    LIMIT 1;

    UPDATE app.projects
    SET status = v_new_status
    WHERE project_id = v_project_id;

    RAISE NOTICE '✅ ТЕСТ 6 ПРОЙДЕН: Обновление в своём сегменте
работает';
    RAISE NOTICE '   • Обновлён проект ID: %', v_project_id;
    RAISE NOTICE '   • Статус изменён: % → %', v_old_status, v_new_status;

    -- Откат изменений
    UPDATE app.projects SET status = v_old_status WHERE project_id =
v_project_id;
END $$;

RESET ROLE;

------------------------------------------------------------
-- Тест 7. Попытка обновить запись с подменой сегмента блокируется
------------------------------------------------------------
DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
    RAISE NOTICE 'ТЕСТ 7: Запрет подмены segment_id при обновлении';
    RAISE NOTICE 'Роль: studio_manager, Сегмент: 1';
    RAISE NOTICE 'Ожидается: ошибка при попытке изменить segment_id на
2';
```

```
END $$;

SET ROLE studio_manager;
SELECT app.set_session_ctx(1, NULL);

DO $$
DECLARE
   v_project_id INTEGER;
BEGIN
   SELECT project_id INTO v_project_id
   FROM app.projects
   WHERE segment_id = 1
   ORDER BY project_id
   LIMIT 1;

   BEGIN
      UPDATE app.projects
      SET segment_id = 2
      WHERE project_id = v_project_id;
      RAISE EXCEPTION '❌ ТЕСТ 7 ПРОВАЛЕН: удалось сменить
segment_id на чужой';
   EXCEPTION WHEN others THEN
      IF SQLSTATE = '42501' OR SQLERRM LIKE '%row-level security%'
THEN
         RAISE NOTICE '✅ ТЕСТ 7 ПРОЙДЕН: Подмена сегмента
заблокирована';
         RAISE NOTICE '   • Проект ID: %', v_project_id;
         RAISE NOTICE '   • Ошибка: %', SQLERRM;
      ELSE
         RAISE EXCEPTION '❌ ТЕСТ 7: Неожиданная ошибка: %',
SQLERRM;
      END IF;
   END;
END $$;

RESET ROLE;

-------------------------------------------------------------
-- Тест 8. Auditor видит все сегменты БЕЗ установки контекста
-------------------------------------------------------------
DO $$
BEGIN
   RAISE NOTICE '';
```

```
    RAISE NOTICE '━━━
```

```
    SELECT COUNT(DISTINCT segment_id), COUNT(*)
    INTO v_sessions_segments, v_sessions_total
    FROM app.recording_sessions;

    -- Проверка таблицы audio_archives
    SELECT COUNT(DISTINCT segment_id), COUNT(*)
    INTO v_audio_segments, v_audio_total
    FROM app.audio_archives;

    -- Проверка таблицы voice_roles
    SELECT COUNT(DISTINCT segment_id), COUNT(*)
    INTO v_voice_roles_segments, v_voice_roles_total
    FROM app.voice_roles;

    -- Валидация результатов
    IF v_projects_segments < 3 THEN
        RAISE EXCEPTION '❌ ТЕСТ 9 ПРОВАЛЕН: auditor видит лишь % сегментов в projects (ожидалось ≥3)', v_projects_segments;
    END IF;

    RAISE NOTICE 'ТЕСТ 9 ПРОЙДЕН: Auditor имеет полный доступ ко всем данным';
    RAISE NOTICE '';
    RAISE NOTICE '
┌──────────────────────┬──────────────┬──────────────┐';
    RAISE NOTICE '│ Таблица              │ Сегментов    │ Всего записей │';
    RAISE NOTICE '
├──────────────────────┼──────────────┼──────────────┤';
    v_line := '│ app.projects         │ ' || rpad(v_projects_segments::TEXT, 11) || ' │ ' || rpad(v_projects_total::TEXT, 12) || ' │';
    RAISE NOTICE '%', v_line;
    v_line := '│ app.contracts        │ ' || rpad(v_contracts_segments::TEXT, 11) || ' │ ' || rpad(v_contracts_total::TEXT, 12) || ' │';
    RAISE NOTICE '%', v_line;
    v_line := '│ app.actors           │ ' || rpad(v_actors_segments::TEXT, 11) || ' │ ' || rpad(v_actors_total::TEXT, 12) || ' │';
    RAISE NOTICE '%', v_line;
    v_line := '│ app.recording_sessions │ ' || rpad(v_sessions_segments::TEXT, 11) || ' │ ' || rpad(v_sessions_total::TEXT, 12) || ' │';
    RAISE NOTICE '%', v_line;
    v_line := '│ app.audio_archives   │ ' || rpad(v_audio_segments::TEXT, 11) || ' │ ' || rpad(v_audio_total::TEXT, 12) || ' │';
```

```
    RAISE NOTICE '%', v_line;
    v_line := '    │ app.voice_roles             │ ' ||
rpad(v_voice_roles_segments::TEXT, 11) || ' │ ' ||
rpad(v_voice_roles_total::TEXT, 12) || ' │';
    RAISE NOTICE '%', v_line;
    RAISE NOTICE '
└──────────────────────────────────────────────────────┴──────────────────────┴──
──────────────────────┘ ';
    RAISE NOTICE '';
    RAISE NOTICE '   Auditor видит данные из % сегментов БЕЗ ограничений
RLS', v_projects_segments;
END $$;

RESET ROLE;

------------------------------------------------------------
-- Тест 9. Корректные операции voice_actor в своём сегменте
------------------------------------------------------------
DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
    RAISE NOTICE 'ТЕСТ 8: Доступ voice_actor к своим контрактам';
    RAISE NOTICE 'Роль: voice_actor, Сегмент: 1, Actor ID: 1';
    RAISE NOTICE 'Ожидается: voice_actor видит только свои контракты в
сегменте 1';
END $$;

SET ROLE voice_actor;
SELECT app.set_session_ctx(1, 1);

DO $$
DECLARE
    v_contracts_count INTEGER;
    v_actors_count INTEGER;
    v_actor_name TEXT;
BEGIN
    SELECT COUNT(*) INTO v_contracts_count FROM app.contracts;
    SELECT COUNT(*) INTO v_actors_count FROM app.actors;

    SELECT first_name || ' ' || last_name INTO v_actor_name
    FROM app.actors WHERE actor_id = 1;
```

```
    IF v_contracts_count = 0 THEN
        RAISE EXCEPTION '❌ ТЕСТ 8 ПРОВАЛЕН: voice_actor не увидел свои
контракты';
    END IF;

    RAISE NOTICE '✅ ТЕСТ 8 ПРОЙДЕН: voice_actor получил доступ к
своим данным';
    RAISE NOTICE '   • Актёр: % (ID: 1)', v_actor_name;
    RAISE NOTICE '   • Доступно контрактов: %', v_contracts_count;
    RAISE NOTICE '   • Доступно актёров: %', v_actors_count;
END $$;

RESET ROLE;

-- ============================================================
-- Итоговая сводка
-- ============================================================
DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
    RAISE NOTICE
'╔═══════════════════════════════════════════════════════════════════
═══════════════════════════════╗';
    RAISE NOTICE '║                    ИТОГОВАЯ СВОДКА ТЕСТИРОВАНИЯ
                                ║';
    RAISE NOTICE
'╠═══════════════════════════════════════════════════════════════════
═══════════════════════════════╣';
    RAISE NOTICE '║  ✅ Все 9 тестов построчной изоляции успешно
пройдены                        ║';
    RAISE NOTICE '║                                                    ║';
    RAISE NOTICE '║  Проверенные функции:                              ║';
    RAISE NOTICE '║  • Изоляция данных между сегментами
                                ║';
    RAISE NOTICE '║  • Запрет кросс-сегментных операций                ║';
    RAISE NOTICE '║  • Валидация доступа ролей к сегментам
                                ║';
    RAISE NOTICE '║  • Проверка принадлежности актёров к сегментам
                                ║';
    RAISE NOTICE '║  • Защита от подмены segment_id                    ║';
```

```
    RAISE NOTICE '║  • Режим аудитора (полный доступ без ограничений
RLS)        ║';
    RAISE NOTICE '║  • Корректная работа RLS политик                    ║';
    RAISE NOTICE
'╚═══════════════════════════════════════════════════════════════════
═══════════════════════════╝';
    RAISE NOTICE '';
END $$;
```